# Fundamentos de Complexidade de Algoritmos

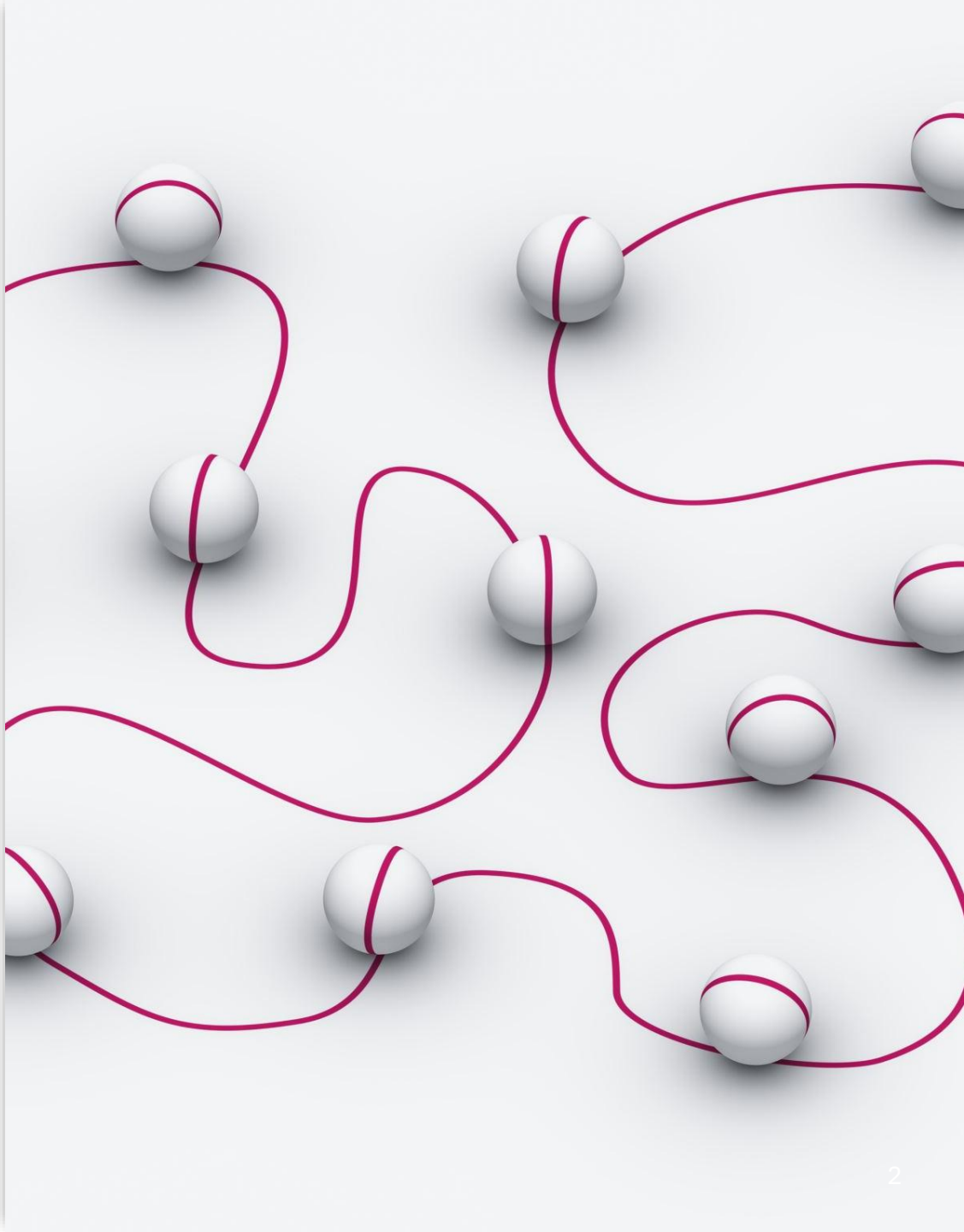
PARTE II

Prof. Ricardo Mesquita

# Ideias Iniciais

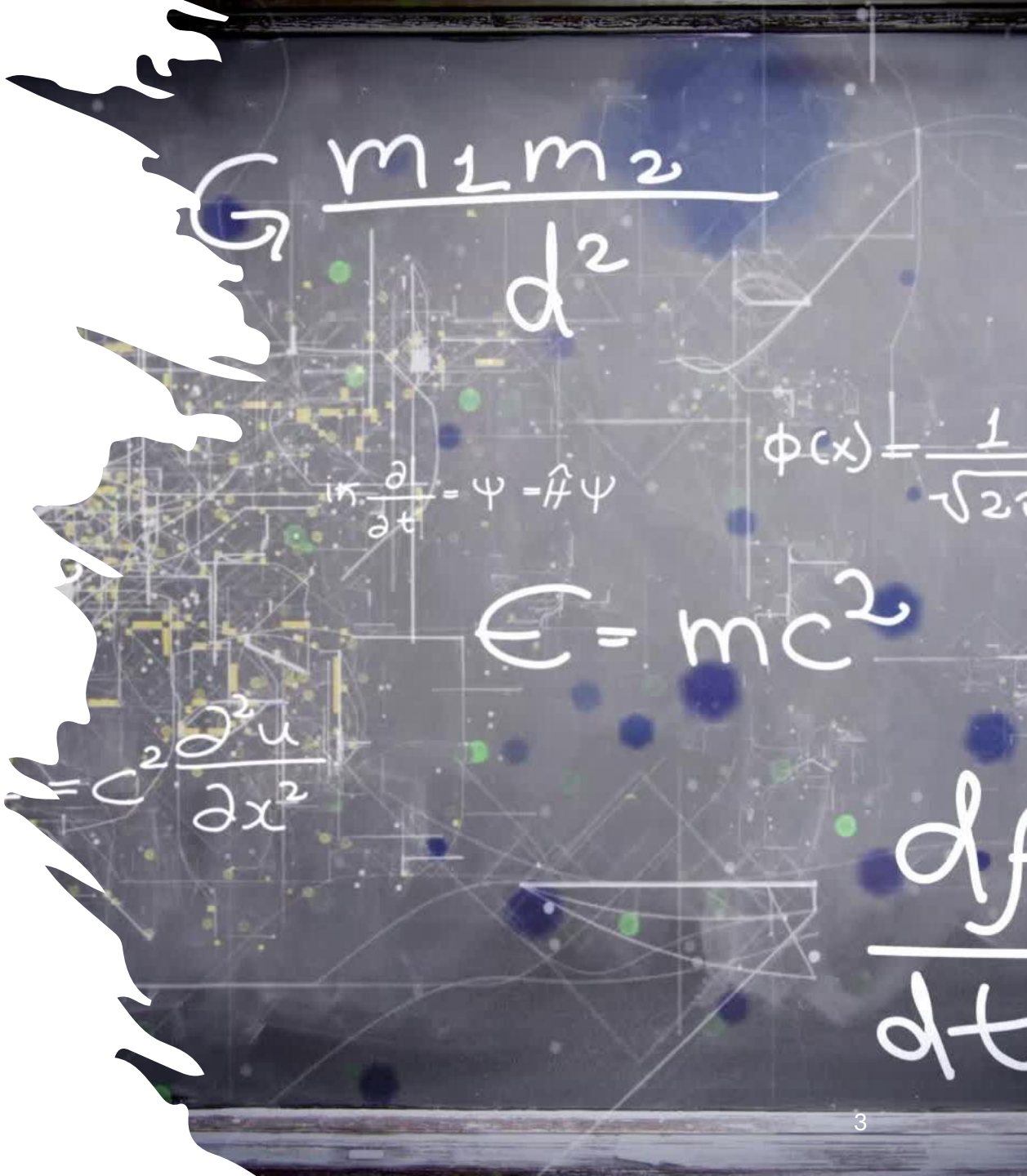
- A ***complexidade assintótica*** é definida pelo crescimento da complexidade para entradas suficientemente grandes.
  - Para um volume grande de dados, a complexidade torna-se mais importante.

- O algoritmo assintoticamente mais eficiente é melhor para todas as entradas, exceto talvez para entradas relativamente pequenas.



# Ideias Iniciais

- Suponha um algoritmo qualquer, cuja complexidade seja dada por $4n + 5$.
- Desejamos simplificar essa expressão para entradas de tamanho grande.
- Para $n$ grande, a parcela linear $4n$ torna-se **dominante** face à parcela constante 5.
    - Portanto, podemos desprezar essa parcela face à primeira!
- A diferença entre $4n + 5$ e $4n$ é 5, apenas 1,25% para $n = 100$, e essa diferença percentual decresce à medida que $n$ cresce.
- Assim, podemos simplificar $4n + 5$ para $4n$.
- Note que a expressão simplificada $4n$ cresce mais rapidamente do que $n$, mas sua razão é uma **constante** 4.
    - Tal razão constante é similar à que ocorre por uma mudança de escala.
    - Desse modo, pode-se simplificar $4n$ para $n$
        - a menos de uma constante multiplicativativa, elas apresentam o ***mesmo crescimento assintótico***.

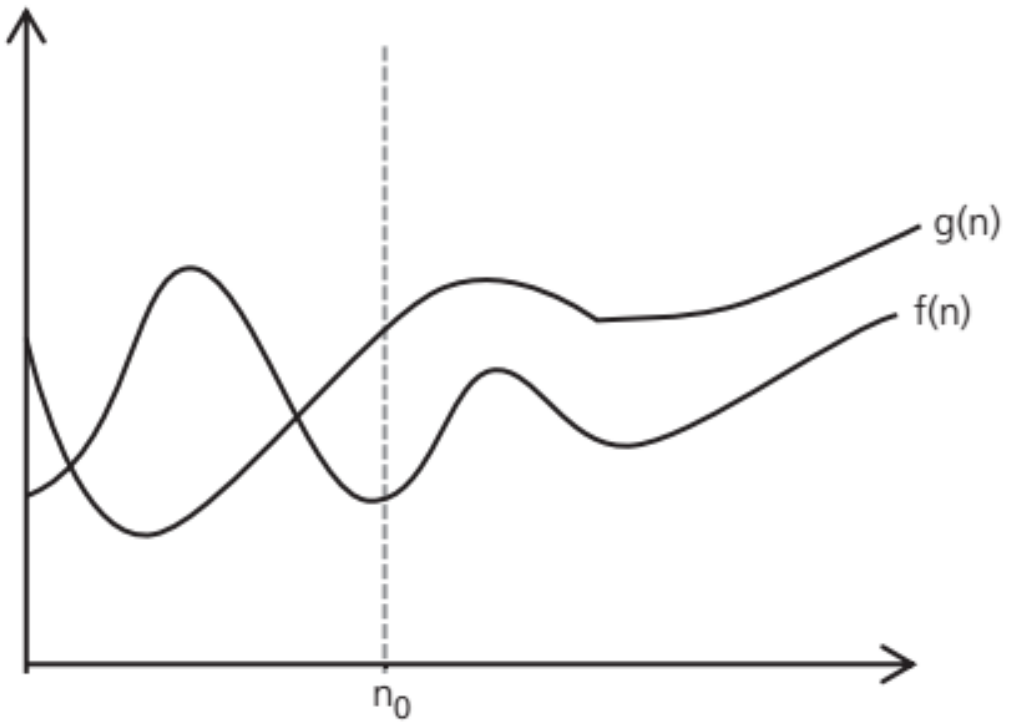


# Cota Assintótica Superior (CAS)

- É uma função que cresce mais rapidamente do que outra, ou seja, permanece acima a partir de certo ponto.
  - Por exemplo, a função quadrática $3n^2$ cresce mais rapidamente do que a linear $n + 5$.
- Define-se que $g$ é uma cota assintótica superior para $f$ se

$$(\exists n_0 \in \mathbb{N})\ (\forall n \geq n_0)\colon f(n) \leq g(n).$$

***Obs***: Uma função que sempre domina outra (como $n^2 \geq n$) é uma CAS, mas isso não é necessário: basta que $g(n)$ domine $f(n)$ para $n$ suficientemente grande.



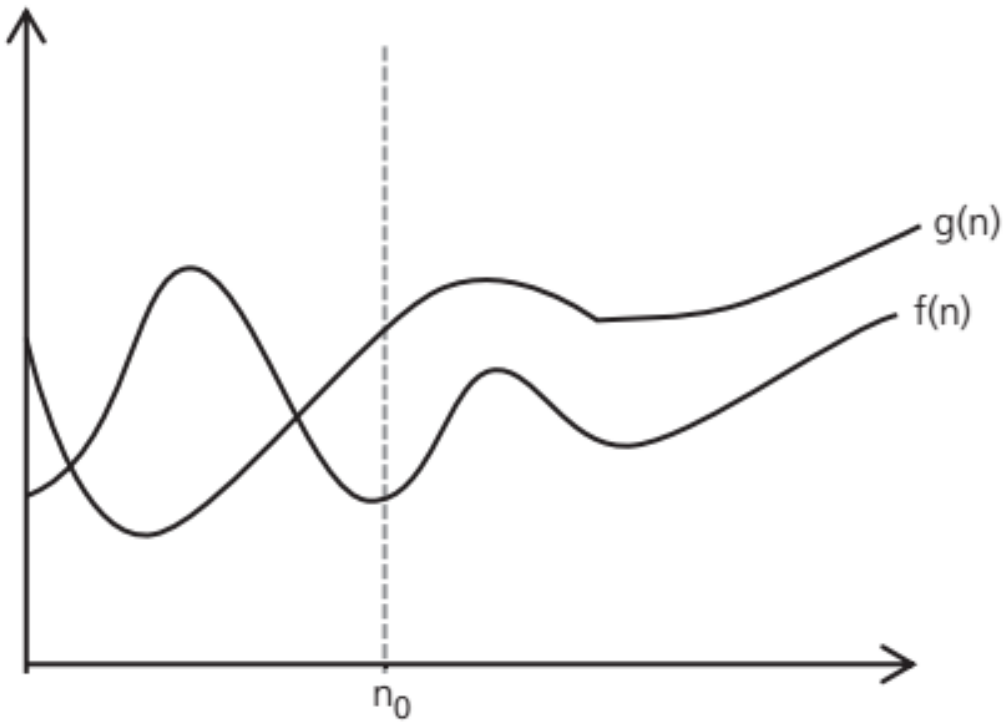
# Notação O

- A notação O define uma cota assintótica superior a menos de constantes.
- Por exemplo, a função quadrática $g(n) = n^2$ cresce mais rapidamente do que a linear $f(n) = 7n + 13$ (a partir de certo ponto – qual?). Dizemos que a linear $f(n)$ é $O(g(n))$
- Define-se que $f(n)$ é $O(g(n))$ se e somente se, para alguma constante $c \in \mathbb{R}_+$: $c\, g(n)$ é CAS para $f(n)$, isto é,

$$(\exists c \in \mathbb{R}_+)\ (\exists n_0 \in \mathbb{N})\ (\forall n \geq n_0)\colon f(n) \leq c\, g(n).$$

# Notação O

- Isso significa que, para *n* suficientemente grande, *g*(*n*) é uma cota superior para *f*(*n*) com um fator constante, isto é, existem constantes *c* e $n_0$ tais que, para valores de *n* não menores que $n_0$, o valor de *f*(*n*) é sempre menor ou igual ao produto *c g*(*n*).



# Exercícios

Para as seguintes funções $f$ e $g$ a seguir, verifique que $f(n)$ é $O(g(n))$, determinando constantes $n_0 \in \mathbb{N}$ e $c \in \mathbb{R}_+$.

1. $f(n) = 5n + 7$ e $g(n) = n^2$
2. $f(n) = 7n + 13$ e $g(n) = 3n^2$
3. $f(n) = 10n$ e $g(n) = n^3$

# Notação O

- Pode-se determinar a ordem de um polinômio a partir de seu grau.
- Por exemplo, considerando o polinômio cúbico $k(n) = n^3 + 3n^2 + 7n + 5$, para $n \geq 1$, temos

$$k(n) = \begin{array}{ccccccc} n^3 & + & 3 \cdot n^2 & + & 7n & + & 5 \\ \| & & \wedge\!| & & \wedge\!| & & \wedge\!| \\ 1 \cdot n^3 & + & 3 \cdot n^3 & + & 7 \cdot n^3 & + & 5 \cdot n^3 \end{array} = 16 \cdot n^3$$

$(1+3+7+5).n^3.$

- **Fato**: Para um polinômio $p(n) = a_m n^m + \ldots + a_1 n + a_0$ de grau m: seu valor absoluto $|\, p(n) \,|$ é $O(n^m)$.

# Notação O

## Demonstração

$$
\begin{aligned}
|\, p(n) &\leq |\, a_m \,| \,.\, n^m + |\, a_{m-1} \,| \,.\, n^{m-1} + \ldots + |\, a_1 \,| \,.\, n + |\, a_0 \,| \\
&= [\, |\, a_m \,| + (\, |\, a_{m-1} \,| \,/\, n\,) + \ldots + (\, |\, a_1 \,| \,/\, n^{m-1}\,) + (\, |\, a_0 \,| \,/\, n^m\,)\,] \,.\, n^m \quad (\text{para } n \neq 0) \\
&\leq [\, |\, a_m \,| + |\, a_{m-1} \,| + \ldots + |\, a_1 \,| + |\, a_0 \,|\,] \,.\, n^m \qquad \text{para } n \geq 1
\end{aligned}
$$

Então, escolhendo $c := |\, a_m \,| + |\, a_{m-1} \,| + \ldots + |\, a_1 \,| + |\, a_0 \,|$ e $n_0 := 1$, tem-se

$$|\, p(n) \,| \leq c \,.\, n^m, \text{ para todo } n \geq n_0. \qquad \blacksquare$$

# Exercícios

1. Considerando o polinômio quadrático $q(n) = 5n^2 + 7n + 3$, mostre que $q(n) = O(n^2)$, determinando constantes $n_0 \in \mathbb{N}$ e $c \in \mathbb{R}_+$ tais que $q(n) \leq c\, n^2$, para todo $n \geq n_0$.
2. Considerando o polinômio cúbico $k(n) = n^3 + 3n^2 + 7n + 5$, verifique se é possível garantir $k(n) \leq 2n^3$, para todo $n \geq 8$.
3. Mostre que relação O é transitiva: se $f(n) = O(g(n))$ e $g(n) = O(h(n))$, então $f(n) = O(h(n))$.
4. Suponha que $f_i = O(g_i)$, para $i = 1, 2$. Mostre ou dê um contraexemplo:

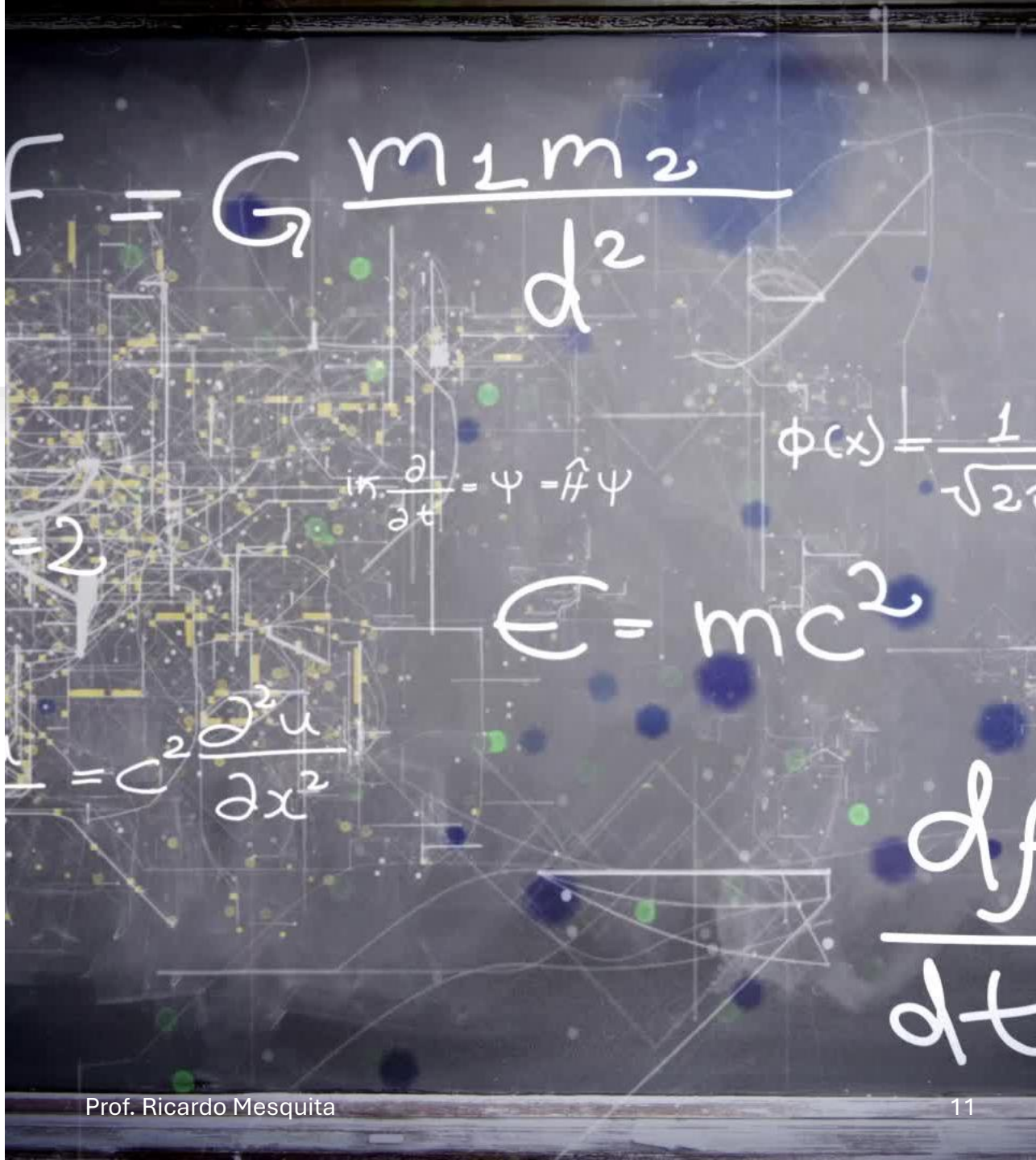
   a) $f_1 + f_2 = O(g_1 + g_2)$;

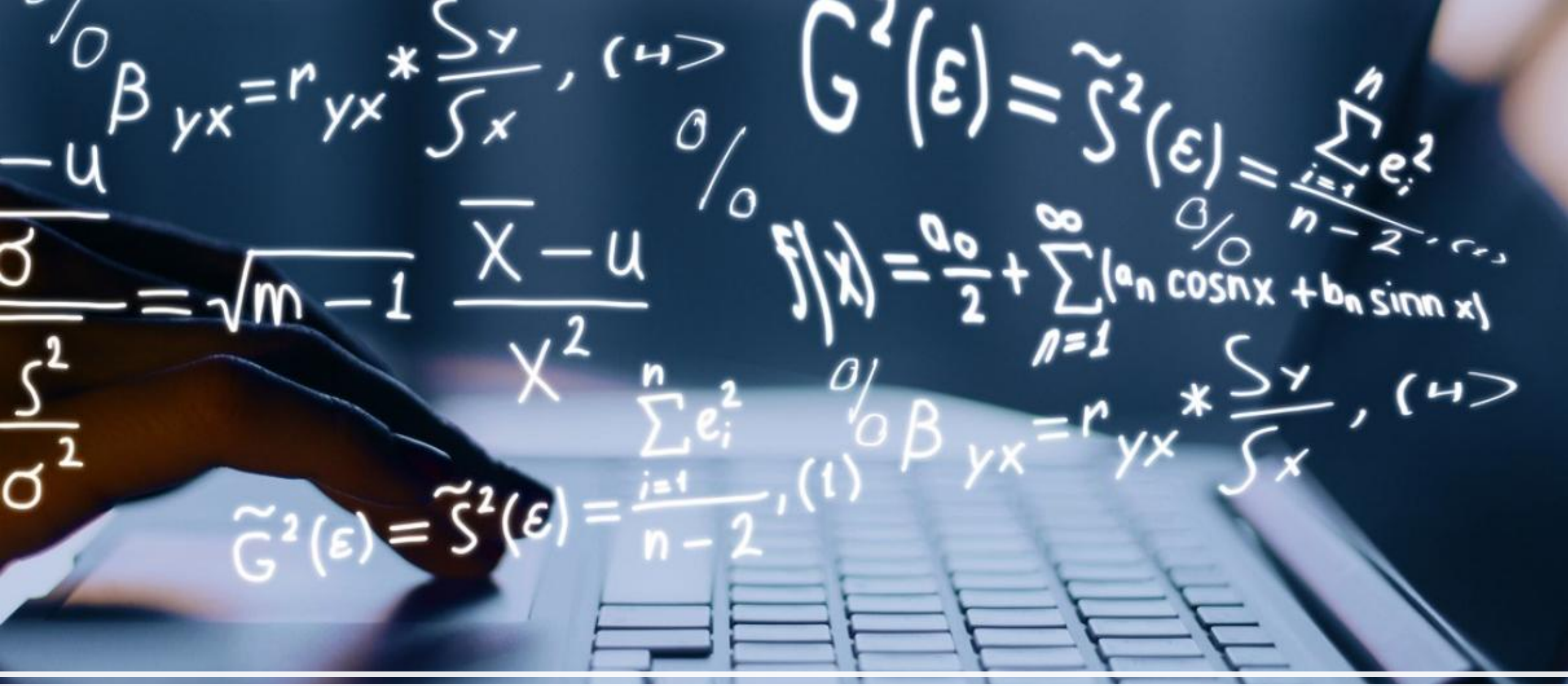
   b) $f_1 \cdot f_2 = O(g_1 \cdot g_2)$;

   c) $2^{f_1} = O(2^{g_1})$.

# Observações

- Os algoritmos de ordem O(1), O($n$), O($n^2$) etc. são polinomiais, isto é, têm complexidade limitada por um polinômio (***complexidade polinomial***).
- Já $2^n$ é uma função não limitada por polinômio, ou seja, não existe m tal que $2^n$ seja O($n^m$). Um algoritmo de ordem O($2^n$), O($3^n$) etc., é dito ***exponencial***.
- Quando o algoritmo é polinomial no valor da entrada (quando a entrada é um polinômio), o algoritmo é dito ***pseudopolinomial***.



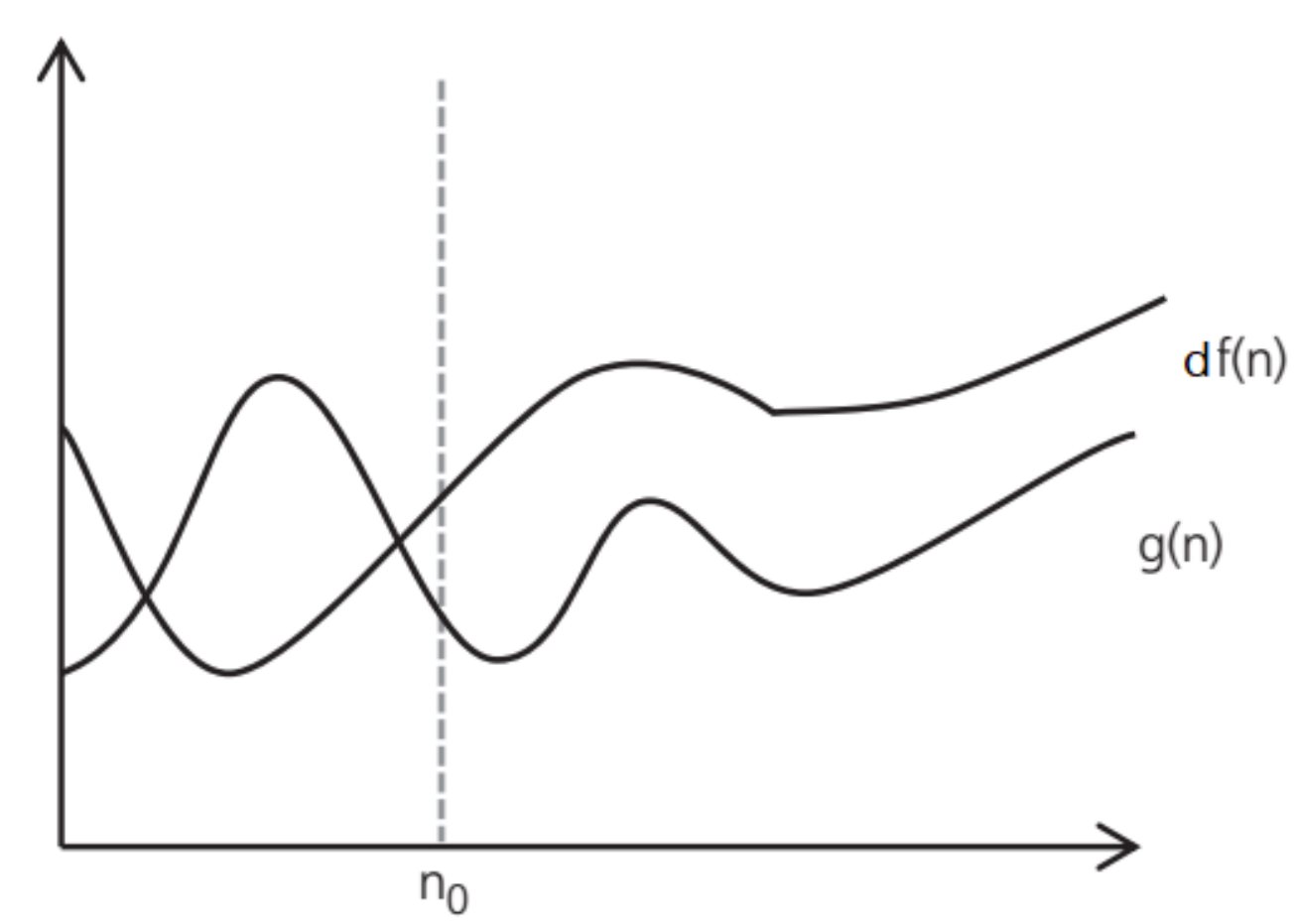


# Outras Ordens Assintóticas

# Notação Ω

- A notação $\Omega$ define uma *cota assintótica inferior* a menos de constantes.
- Por exemplo, a função cúbica $g(n) = 7n^3 + 5$ cresce menos rapidamente do que a exponencial $f(n) = 2^n$ (a partir de certo ponto).
- Nesse caso, diz-se que a exponencial $f(n)$ é $\Omega(g(n))$.
- De maneira geral, define-se f(n) é $\Omega(g(n))$ se e somente se para alguma constante $d \in \mathbb{R}_+$: $d\,f(n)$ é CAS para $g(n)$, isto é,

$$(\exists d \in \mathbb{R}_+)(\exists n_0 \in \mathbb{N})(\forall n \geq n_0)\colon d\,f(n) \geq g(n).$$

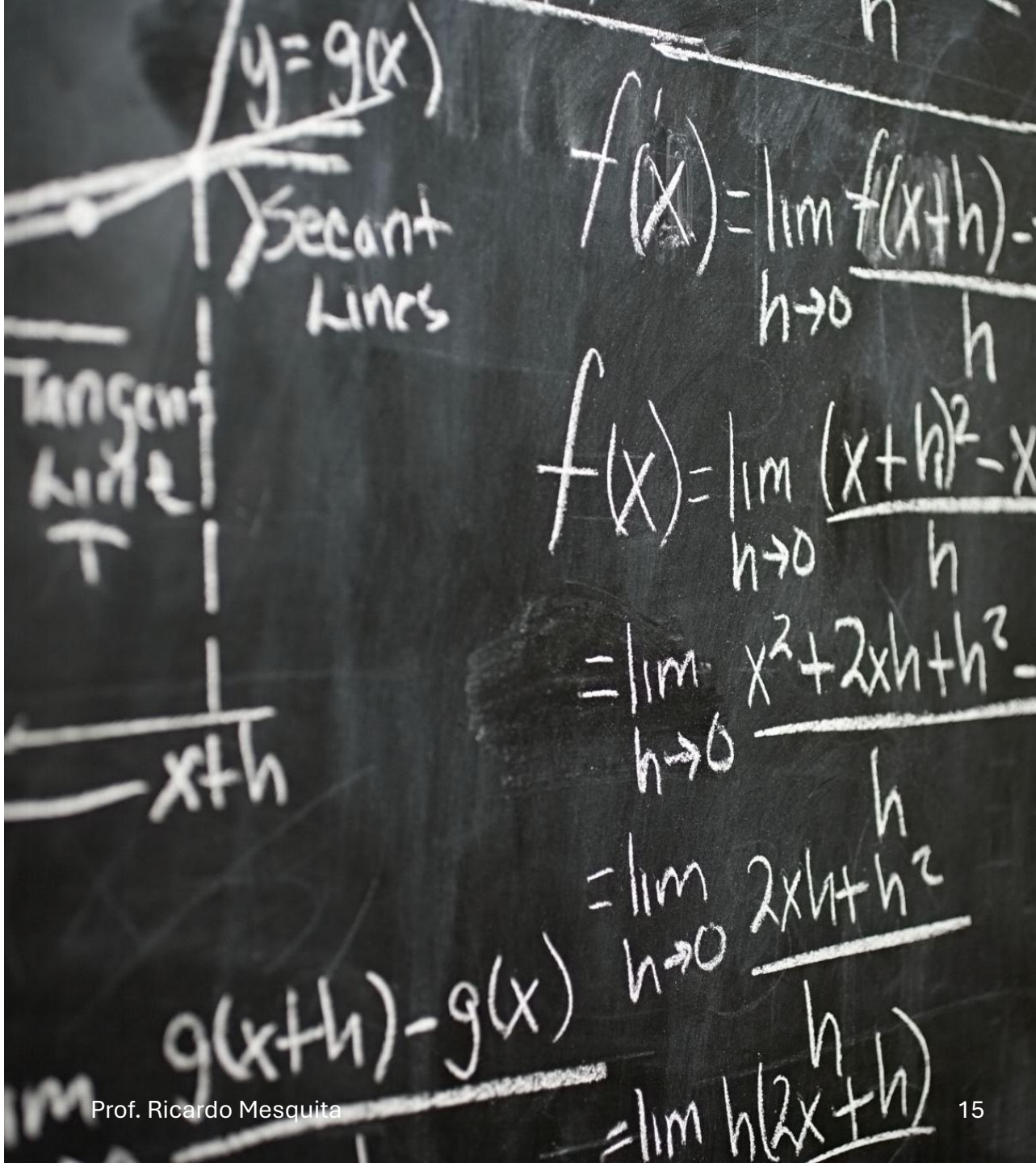



# Notação $\Omega$

- Isso significa que existem constantes $d \in \mathbb{R}_+$ e $n_0 \in \mathbb{N}$ tais que, para valores de $n$ iguais ou superiores a $n_0$, o valor de $g(n)$ é sempre menor ou igual ao produto $d\,f(n)$.

# Exemplo

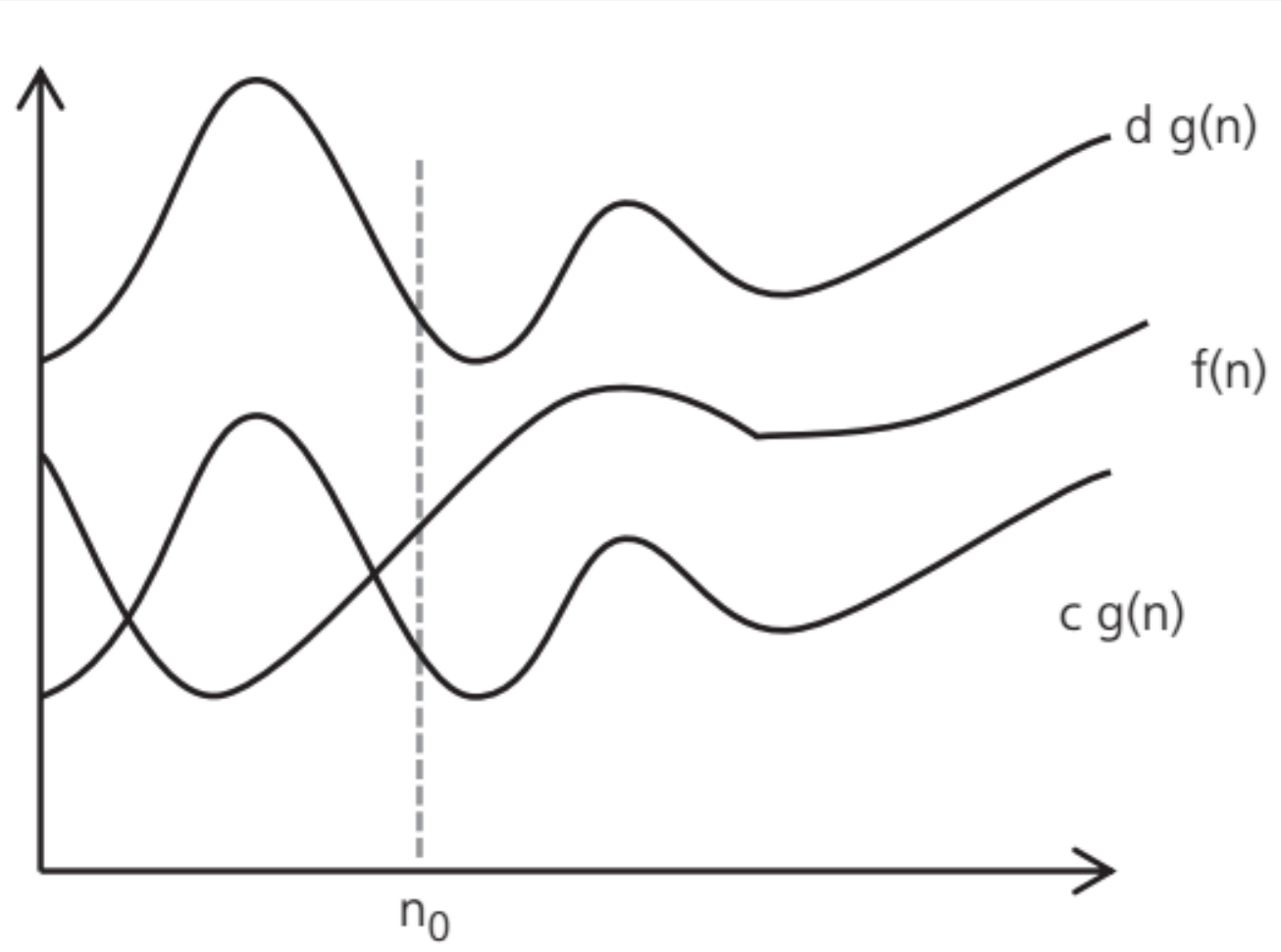
- Considere as funções cúbica $f(n) = n^3$ e quadrática $g(n) = 6n^2 + 10$.
- Para $n \geq 4$, temos $g(n) = 6n^2 + 10 \leq 6n^2 + 16 \leq 6n^2 + n^2 = 7n^2 \leq 9n^2 \leq 3n^3 = 3\,f(n)$.
- Assim, com $n_0 = 4$ e $d = 3$, vemos que para todo $n \geq n_0$, temos $g(n) \leq d\,f(n)$.
- Logo, $f(n)$ é $\Omega(g(n))$.



# Notação $\Theta$

- A notação $\Theta$ define um *limite assintótico exato*, a menos de constantes.
- Por exemplo, as funções quadráticas $f(n) = 7n^2 + 13$ e $g(n) = n^2 + 3$ crescem com a mesma rapidez (a partir de certo ponto). Diz-se que $f(n)$ é $\Theta(g(n))$. Em geral, diz-se $f(n)$ é $\Theta(g(n))$ se e somente se

$$(\exists c, d \in \mathbb{R}_+)(\exists n_0 \in \mathbb{N})(\forall n \geq n_0)\colon c\, g(n) \leq f(n) \;\&\; f(n) \leq d\, g(n).$$

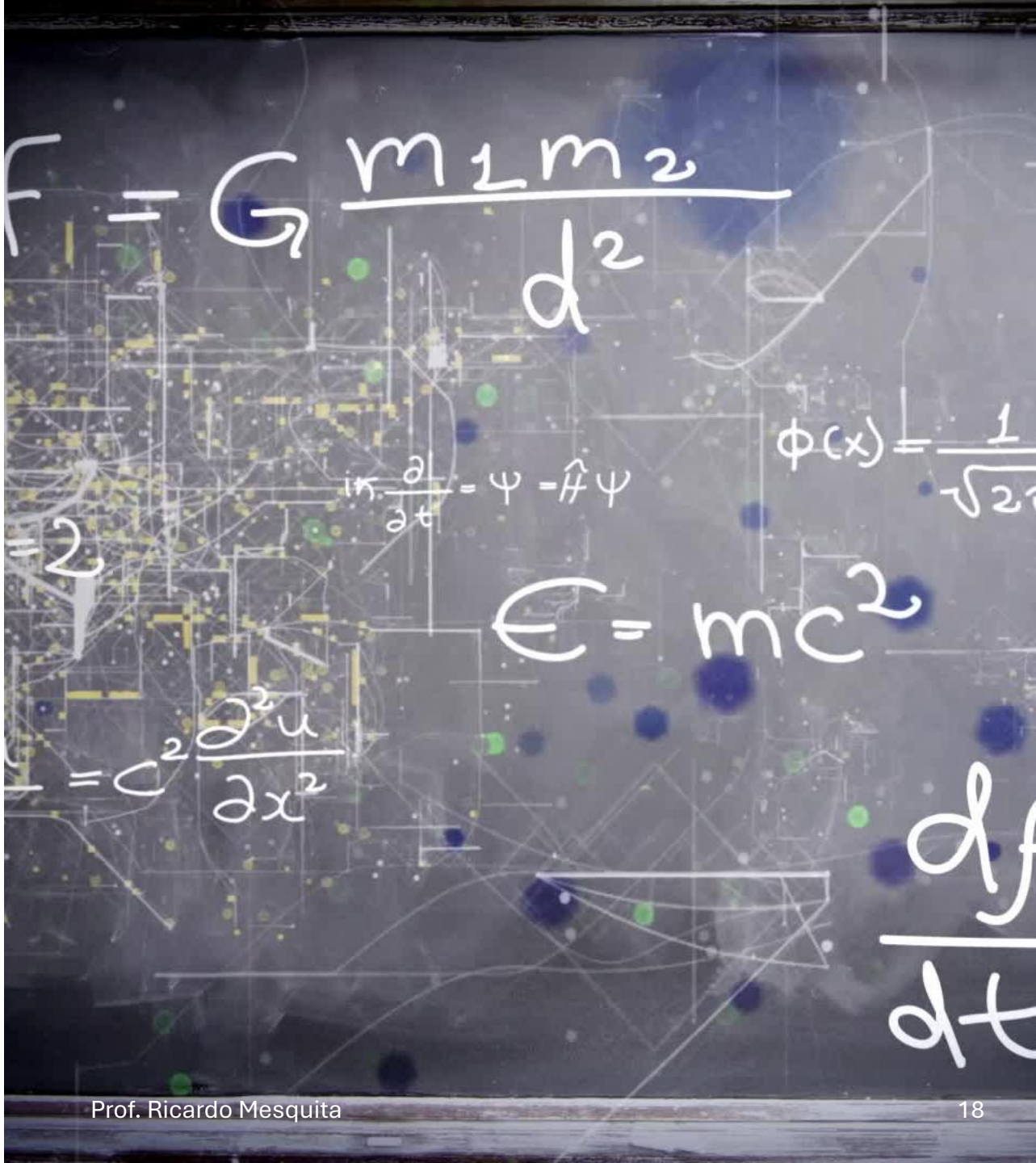


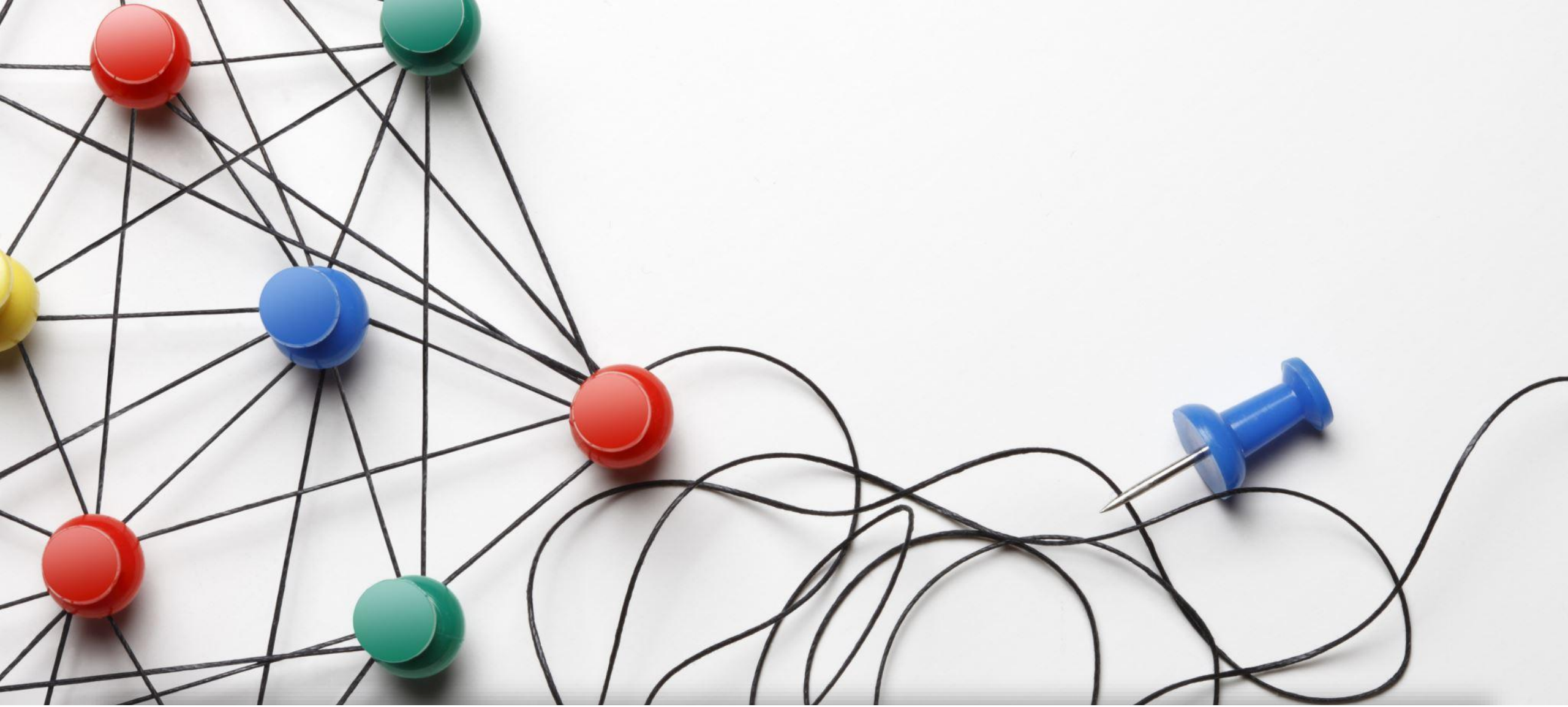

# Notação Θ

- Isto significa que existem constantes *c*, *d* (reais positivas) e $n_0 \in \mathbb{N}$tais que, para todo valor de *n* maior ou igual a $n_0$, o valor de *f*(*n*) está entre os produtos *c* *g*(*n*) e *d* *g*(*n*).

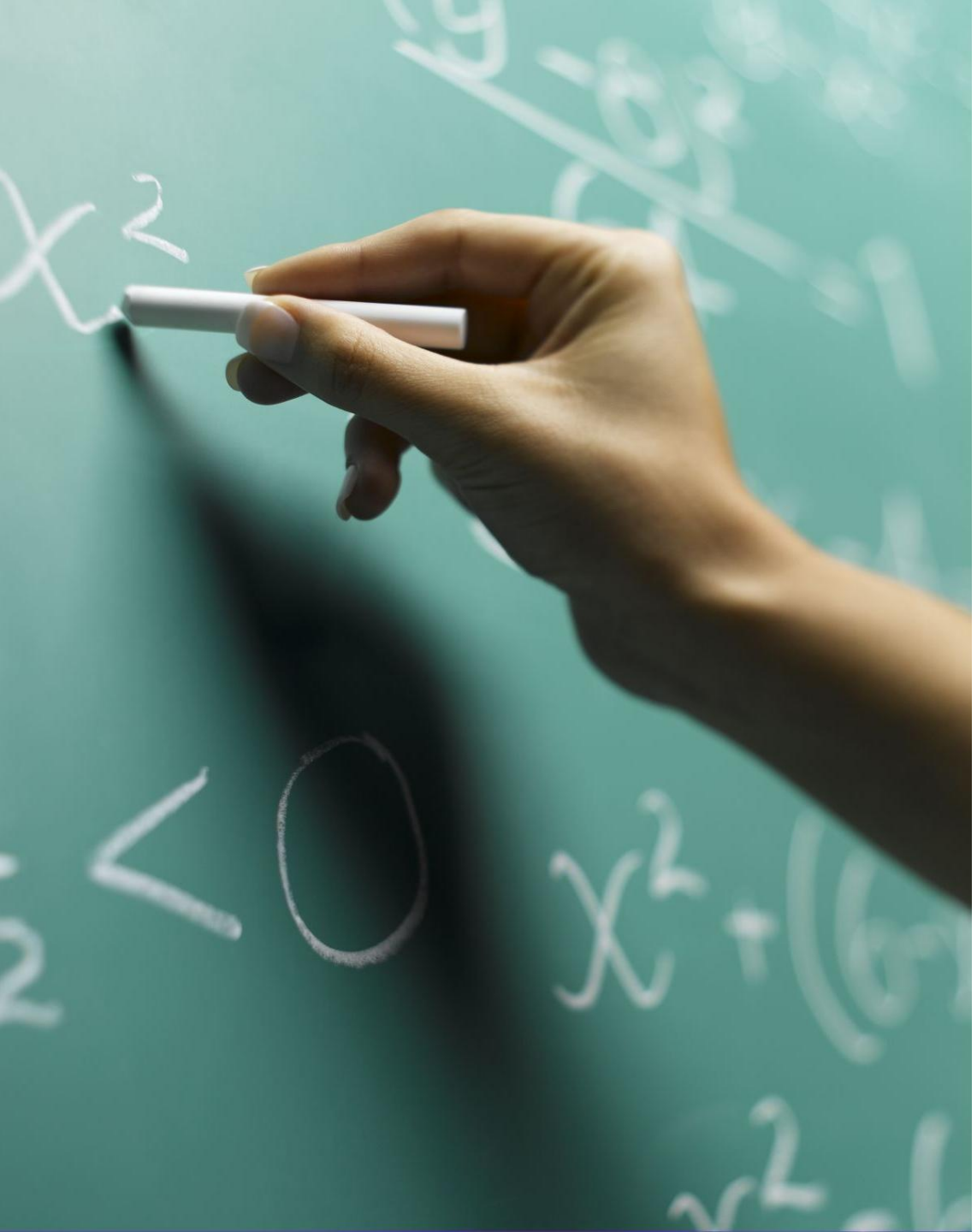
# Exemplo

- Considere as funções lineares $f(n) = 7n + 3$ e $g(n) = 2n + 5$.
- Por um lado, para $n \geq 3$, temos $f(n) = 7n + 3 \leq 7n + n = 8n \leq 8(2n + 5) = 8\, g(n)$.
- Por outro lado, para $n \geq 5$, temos $g(n) = 2n + 5 \leq 2n + n = 3n \leq 3(7n + 3) = 3\, f(n)$.
- Assim, tomando $c = 1/3$, $d = 8$ e $n_0 \geq$ Máx{3, 5}, $n_0 = 6$, vemos que para todo $n \geq n_0$, temos $c\, g(n) \leq f(n)$ e $f(n) \leq d\, g(n)$.
- Logo, $f(n)$ é $\Theta(g(n))$.





# Conceitos de Complexidade Pessimista

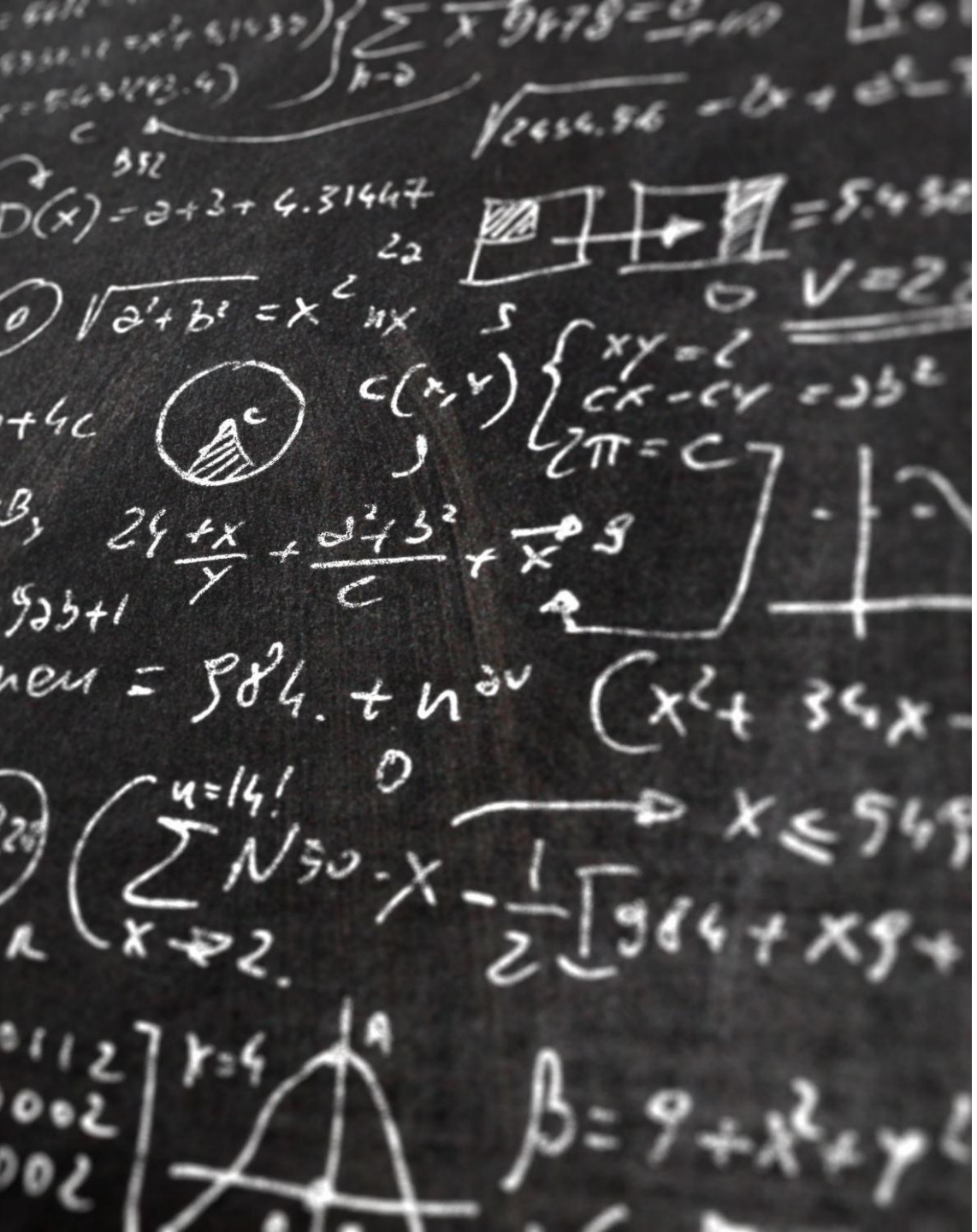


# Introdução

- Apesar de a complexidade ser uma medida muito particular à classe do algoritmo que está sendo analisado (pois depende da medida de tamanho e das operações fundamentais), alguns aspectos do cálculo da complexidade não dependem do que faz o algoritmo, mas somente da sua estrutura, e assim ***podem ser generalizados***.
- Entre as medidas de complexidade, a complexidade no pior caso é o critério de avaliação mais utilizado.

# Componentes de Algoritmos

- O *desempenho* de um algoritmo pode ser obtido a partir dos desempenhos das suas várias partes.
- De maneira similar, a *complexidade* de um algoritmo poderá ser obtida a partir das complexidades das suas componentes.
- Em geral, cada componente executada dá sua contribuição para o desempenho e para a complexidade do algoritmo.



# Componentes de Algoritmos

- Torna-se conveniente classificar as componentes de um algoritmo em *conjuntivas* ou *disjuntivas* conforme sejam sempre executadas ou não.
- Uma componente de um algoritmo será dita **conjuntiva** quando ela é sempre executada em qualquer execução do algoritmo.
- Por outro lado, chamaremos de **disjuntivas** as componentes que são executadas em algumas execuções do algoritmo, dependendo do valor da entrada.

# Exercício

Analise a complexidade do algoritmo ao lado:

```
var    numero, decimal: inteiro
       binario: caracter
inicio
   imprima  "Digite um numero inteiro"
   leia  numero
   decimal ← numero
   enquanto (numero > 0) faça
        se (numero mod 2 = 0)
        então
             binario ← "0" + binario
        senão
             binario ← "1" + binario
        fim_se
            numero ← numero div 2
    fim_enquanto
    imprima  "O numero ", decimal, " em binario = ", binario
fim
```

```
Algoritmo (V, n)
    k = n-1
    para i = 1 até n faça
        j = 1
        enquanto j <= k faça
            se V[j] > V[j+1] então
                aux = V[j]
                V[j] = V[j+1]
                V[j+1] = aux
            j = j+1
        k = k-1
```

# Exercício

- Analise a complexidade do algoritmo ao lado.

# Exercício

- Implemente uma solução para o problema "*The 3n + 1 Problem*".
- Acesse ***onlinejudge.org***

https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=36

- Faça a análise da complexidade de sua solução.

# Até a Próxima Aula!!

Prof. Ricardo Mesquita